

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #13
Hamiltonian cycle

Computer Science

Agenda

- **Hamiltonian cycle**
 - What it is
 - Class of the problem
 - How (not to) solve it
 - How to address the issue
 - NP complete/hard problems can be addressed

Hamiltonian cycle

- A Hamiltonian cycle is a graph cycle (a closed loop) that visits every vertex in a graph exactly once, returning to the starting vertex
- Consider the undirected weighted graph:

edge (a, b, 7) .

edge (a, c, 1) .

edge (a, d, 8) .

edge (b, c, 2) .

edge (b, f, 5) .

edge (b, e, 10) .

edge (c, d, 3) .

edge (d, f, 4) .

edge (d, e, 8) .

edge (f, e, 3) .

is_edge (X, Y, W) :-

edge (X, Y, W) ;

edge (Y, X, W) .

Hamiltonian cycle (NP complete/NP hard problem)

```
[q] ?- hamilton(6,a,Cycle,Cost).
```

```
hamilton(N,X,Cycle,Cost):-
```

```
    N1 is N-1,
```

```
    try(N1,X,X,[X],0,Cycle,Cost).
```

```
try(N, X, Y, PPath, PCost, Cycle, Cost):-
```

```
    N > 0,
```

```
    N1 is N-1,
```

```
    is_edge(Y, Z, W),
```

```
    not(member(Z, PPath)),
```

```
    PCost1 is PCost + W,
```

```
    try(N1, X, Z, [Z|PPath], PCost1, Cycle, Cost).
```

```
try(0, X, Y, PPath, PCost, [X|PPath], Cost):-
```

```
    is_edge(Y, X, W),
```

```
    Cost is PCost + W.
```

Hamiltonian cycle (NP complete / NP hard problem)

- Should not be addressed straight away unless the space is VERY small (how small? Size ~ 20 ?)
- What should be done?

Hamiltonian cycle with branch and bound

- Branch and bound
- Estimate the cost of the path in 2 steps, as follows:
 - $\phi I = \psi i + X_i$
 - ϕI = cost of the path from Start to Stop via the *intermediate* node
 - ψi = cost of the path from Start to the *intermediate* node
 - X_i = cost of the path from the *intermediate* node to Stop
 - They are estimated as: $E[\phi i] = E[\psi i] + E[X_i]$
 - $E[\phi i]$ = estimate cost of the path from Start to Stop via the *intermediate* node
 - $E[\psi i]$ = estimate cost of the path from Start to the *intermediate* node
 - $E[X_i]$ = estimate cost of the path from the *intermediate* node to Stop
- $E[\psi i] = \psi I$
- $E[X_i]$ = the better the estimate, more we narrow the search space (bound more effective). Even a rough estimate can help. Even just 0 proves good.
- The approach (similar to `bf_search`) is also using 2 argument lists:
 - Candidate list – potential path to follow (it is a regular list; not incomplete)
 - Expanded list – part of paths already covered (from *start* to *current* node; none complete to objective)

Hamiltonian cycle with branch and bound

- A structure will be used to estimate costs:
`% n(Node, PathLength, EstimatedCost)`
- The Cand lists looks like: `[n(X,Len,Fi)|Cy]`
 X = node explored
 F_i = estimate value of φ_i $E[\varphi_i] = E[\psi_i] + E[X_i] = \psi_i + 0$
 Len = number of nodes so far, from Start to current (X)
 Cy = parent 'node' = actually the whole path from parent to Start. It is a candidate list!
- Helpers:
`[n(X,Len,Cost) | _] .` % Accesses the first element in the Candidate list
`n(_,Len,_) .` % **Accesses** the **length** of an element in the Candidate list
`n(X,_,_)` % **Accessing** the **node**

Computer Science

Hamiltonian cycle with branch and bound

```

hamilton(N,X,Cycle):-
    search(N,X,[[n(X,0,0)]],[],Cycle).
% searches for N nodes to end in X the way; put in Cands a path containing just one element: [n(X,0,0)]
% start node X, estimated weight of ham E[Xi]=0, number of nodes 0, NO parent. Expanded list empty.

search(N,_,_,[Cycle|_],Cycle):-
    Cycle=[n(_,N,_)|_],!.
search(N,X,[C|Cands],Exp,Cycle):-
    expand(N,X,C,Cands,NewCands),
    search(N,X,NewCands,[C|Exp],Cycle).

% stop with path the way in front of Expanded
% if its length equals the number of nodes in graph

% if not, expand best node so far= the element in front
% and continue after expansion with the new Cands

% 3rd arg is the best candidate list which is expanded,
% take first node (last added) and find neighbours
expand(N,X,[n(Y,Len,Fi)|Cy],_,_):-
    is_edge(Y,Z,W),           % nondeterministically take Z, first neighbor of Y (eventually all of them) and weight W
    (not(member_path(Z,Cy));(Len is N-1,Z=X)), % should NOT be already processed
    % just if it is the source
    FiW is Fi+W,Len1 is Len+1, % estimate the new parameters
    assertz(node(n(Z,Len1,FiW))), % put it in the akb
    fail. % backtrack to a new neighbor of Y
expand(_,_,C,Cands,NewCands):-
    collect(C,Cands,NewCands). % start collecting from akb and place in NewCand

```


Hamiltonian cycle with branch and bound

```

collect(C, Cands, NewCands) :- % L is the parent, a whole path. Is Y in Candidate from clause 2 in search
    retract(node(Node)), !, % take top from akb -> Node=n(Node, Len, Cost)
    ins_ord_list([Node|C], Cands, IntCands), % add in Cands with the whole path
    collect(C, IntCands, NewCands). % continue with the Intermediate Candidates
collect(_, Cands, Cands).

ins_ord_list(X, [H|T], [H|R]) :-
    X = [n(_,_,Xfi)|_], % Extract estimated cost of X
    H = [n(_,_,Yfi)|_], % Extract estimated cost of Y
    Xfi >= Yfi, !, % Compares the E[ψi] functions for nodes X and Y
    ins_ord_list(X, T, R).
ins_ord_list(X, T, [X|T]) :- !. % adds an element

member_path(X, [n(X,_,_)|_]). % checks if a node is in a list
member_path(X, [_|T]) :- member_path(X, T).

```

Hamiltonian cycle with branch and bound

Complete code

```
% hamilton /3 (+NumberOfNodes, +Source, -Cycle)
hamilton(N,X,Cycle):-
    search(N,X,Cycle,[[n(X,0,0)]],[[]]).
```

```
search(N,X,_,[Cycle|_],Cycle):-
    Cycle=[n(_,N,_)|_],!.
search(N,X,[C|Cands],Exp,Cycle):-
    expand(N,X,C,Cands,NewCands),
    search(N,X,NewCands,[C|Exp],Cycle).
```

```
expand(N,X,[n(Y,Len,Fi)|Cy],_,_):-
    is_edge(Y,Z,W),
    (
        not(member_thread(Z,Cy));
        (Len is N-1,Z=X)
    ),
    FiW is Fi+W,
    Len1 is Len+1,
    assertz(node(n(Z,Len1,FiW))),
    fail.
expand(_,_,C,Cands,NewCands):-
    collect(C,Cands,NewCands).
```

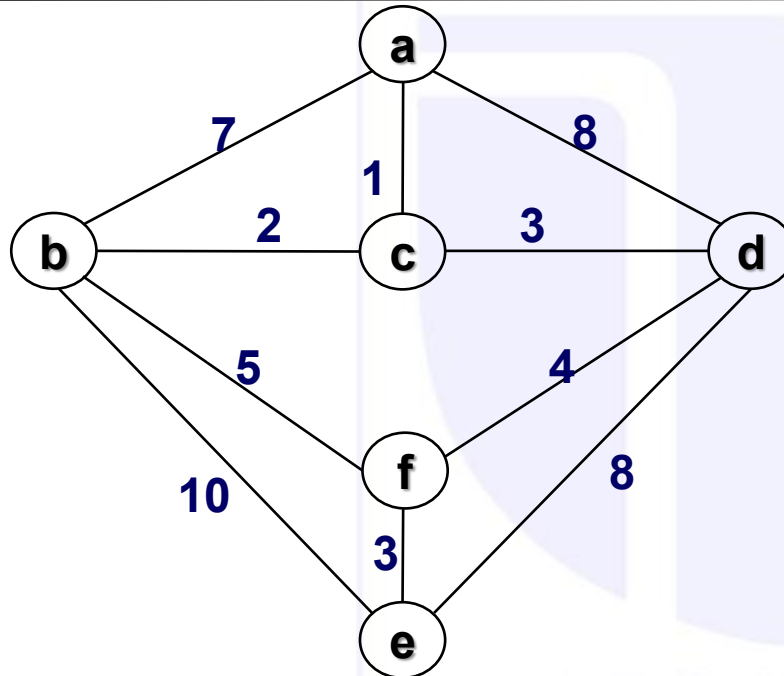
```
collect(C,Cands,NewCands):-
    retract(node(N)),!,
    ins_ord_list([N|C],Cands,IntCands),
    collect(C,IntCands,NewCands).
collect(_,Cands,Cands).
```

```
ins_ord_list(X,[H|T],[H|R]):-
    X = [n(_,_,Xfi)|_],
    H = [n(_,_,Yfi)|_],
    Xfi>=Yfi,!,
    ins_ord_list(X,T,R).
ins_ord_list(X,T,[X|T]):-!.
```

```
member_path(X,[n(X,_,_)|_]).
member_path(X,[_|T]):- member_path(X,T).
```

```
?- hamilton(6, a, Path)
?- search(6,a,[[n(a,0,0)]],[[]],Way).
```

Example



Cand – ordered list of candidate partial paths, waiting expansion

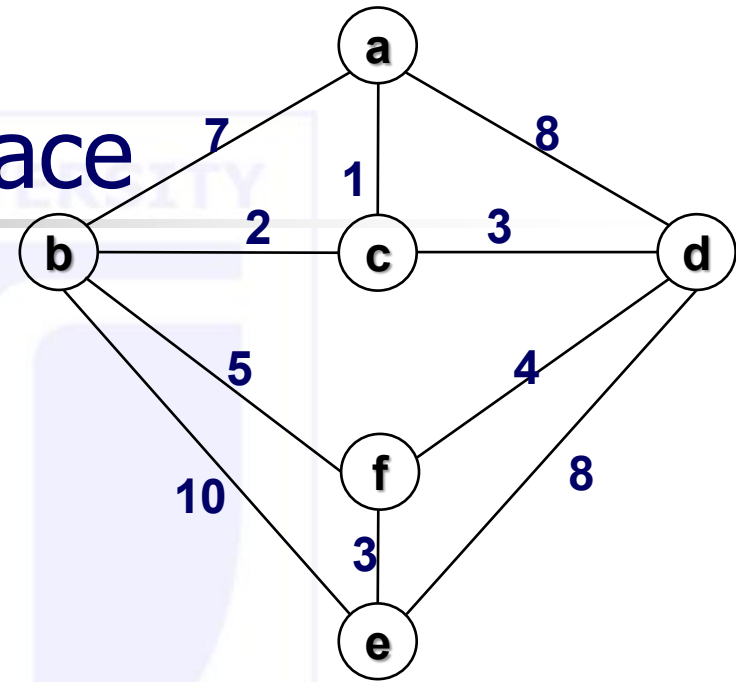
Exp – accumulator list of expanded partial paths (also ordered)

?- search(6,a,[[n(a,0,0)]],[],Cycle).

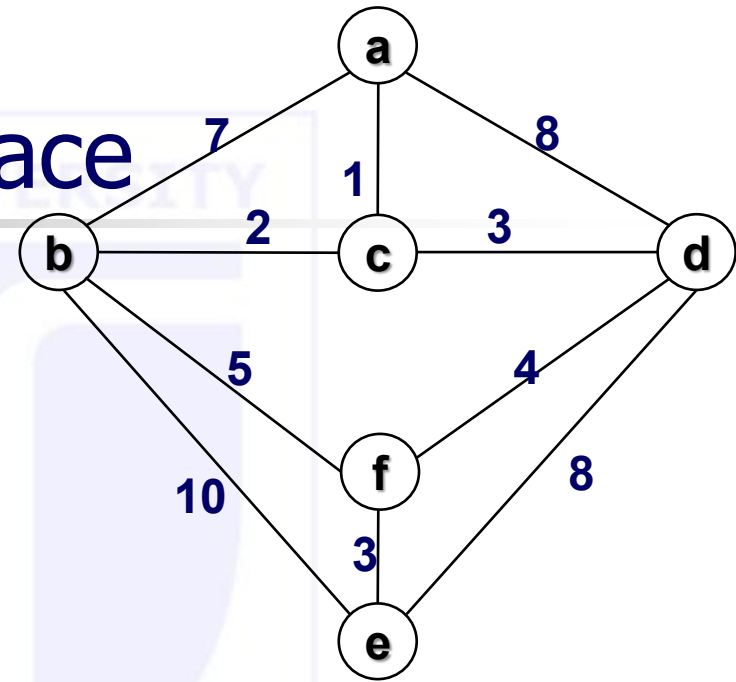
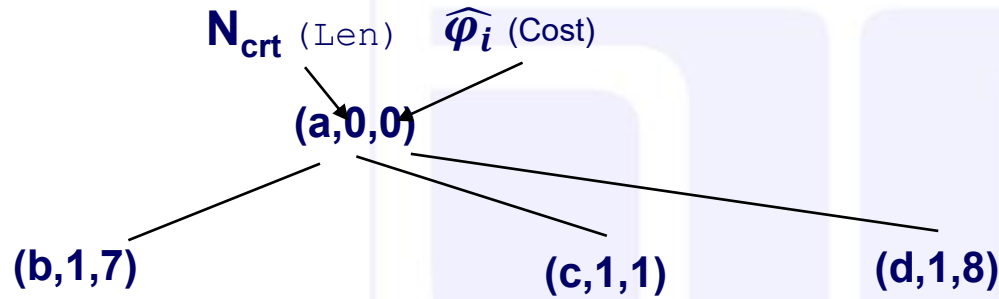
result list; at the end (crt length = N) it is first element of **Exp**

Example – search space

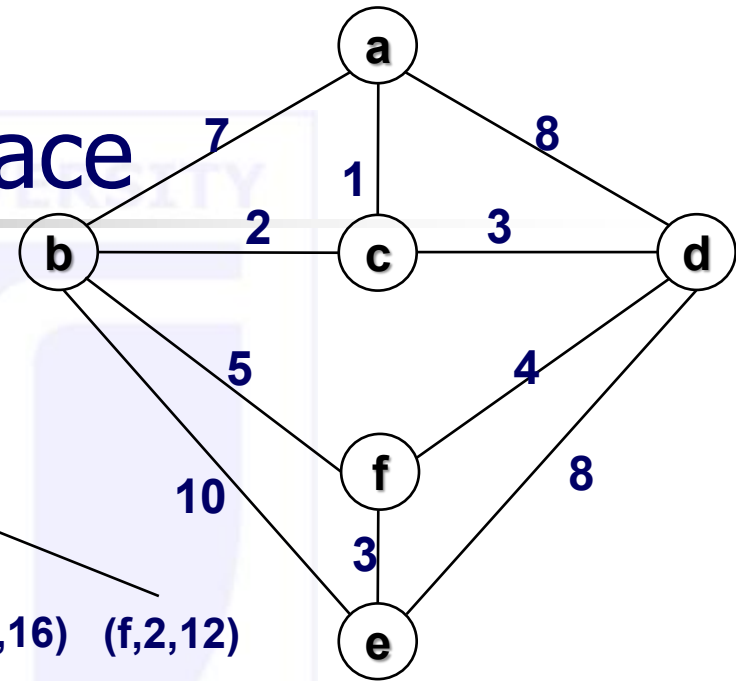
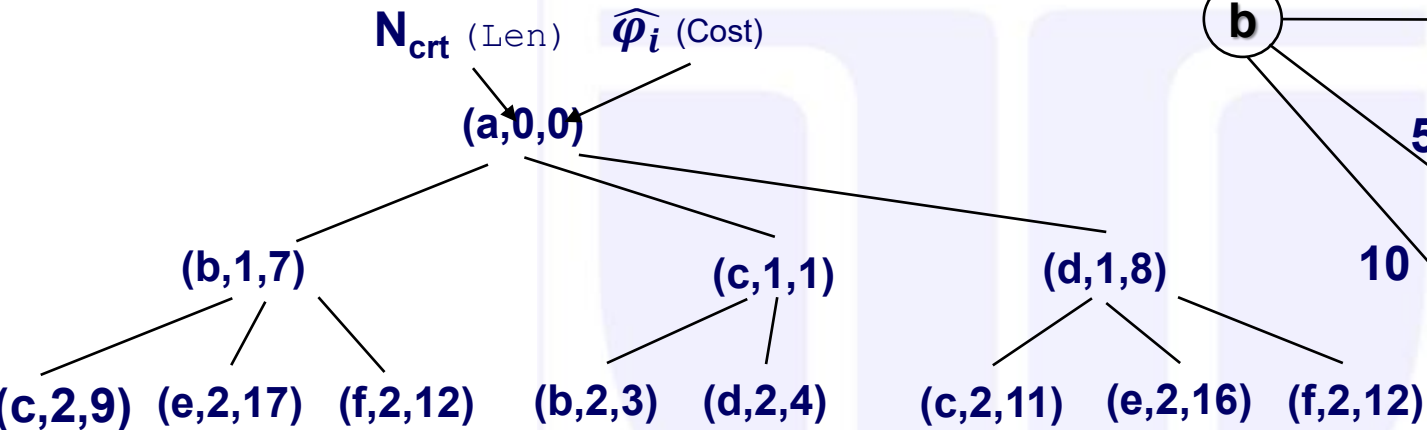
N_{crt} (Len) $\widehat{\varphi}_i$ (Cost)
 (a,0,0)



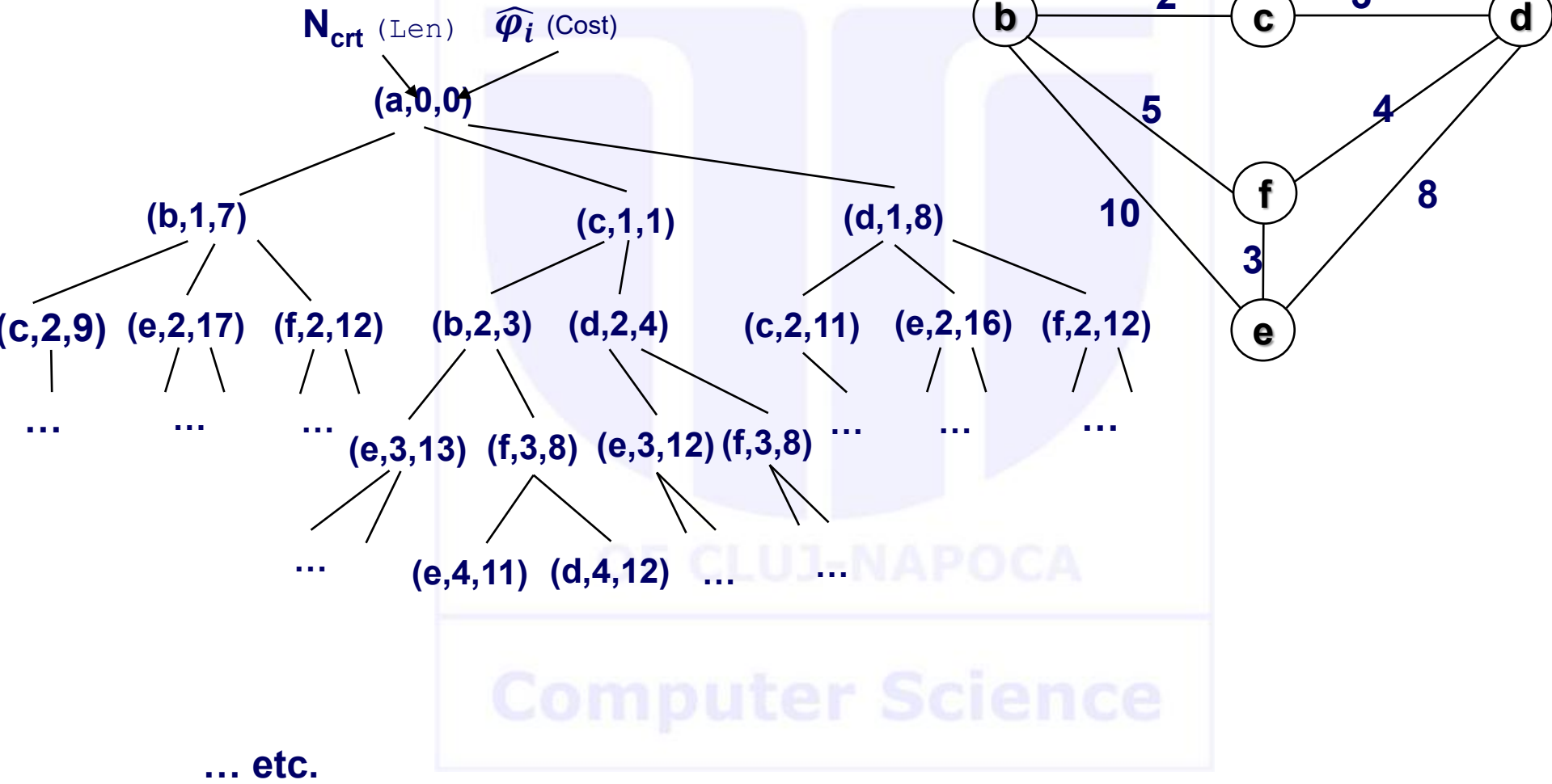
Example – search space



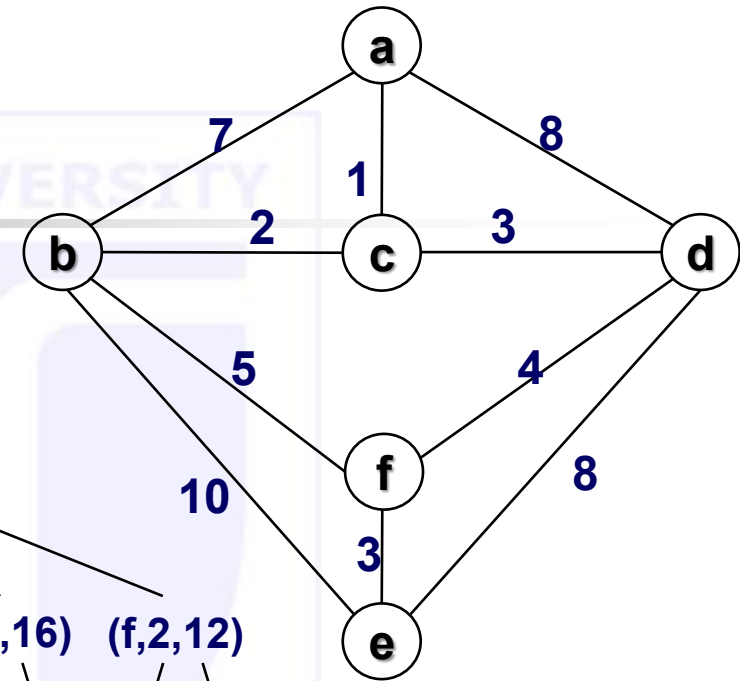
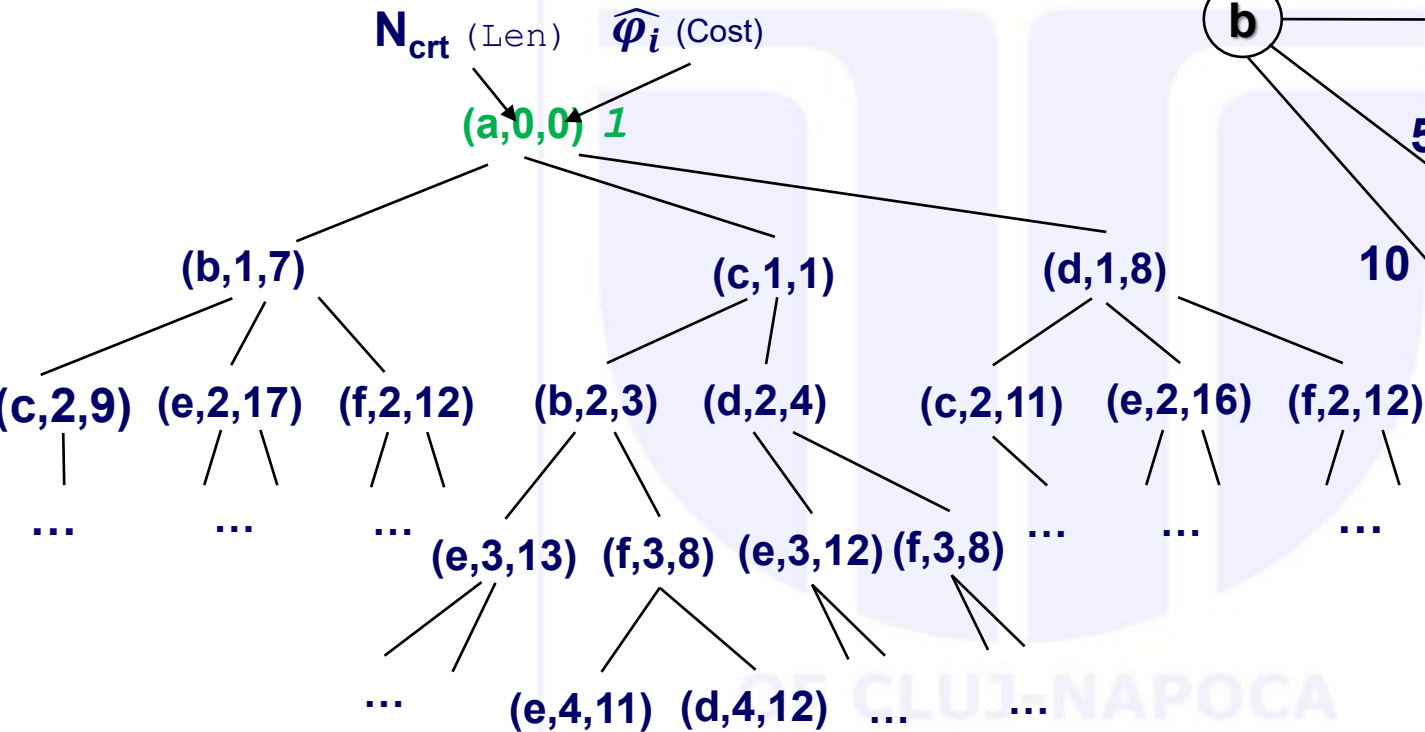
Example – search space



Example – search space



Example – traversal

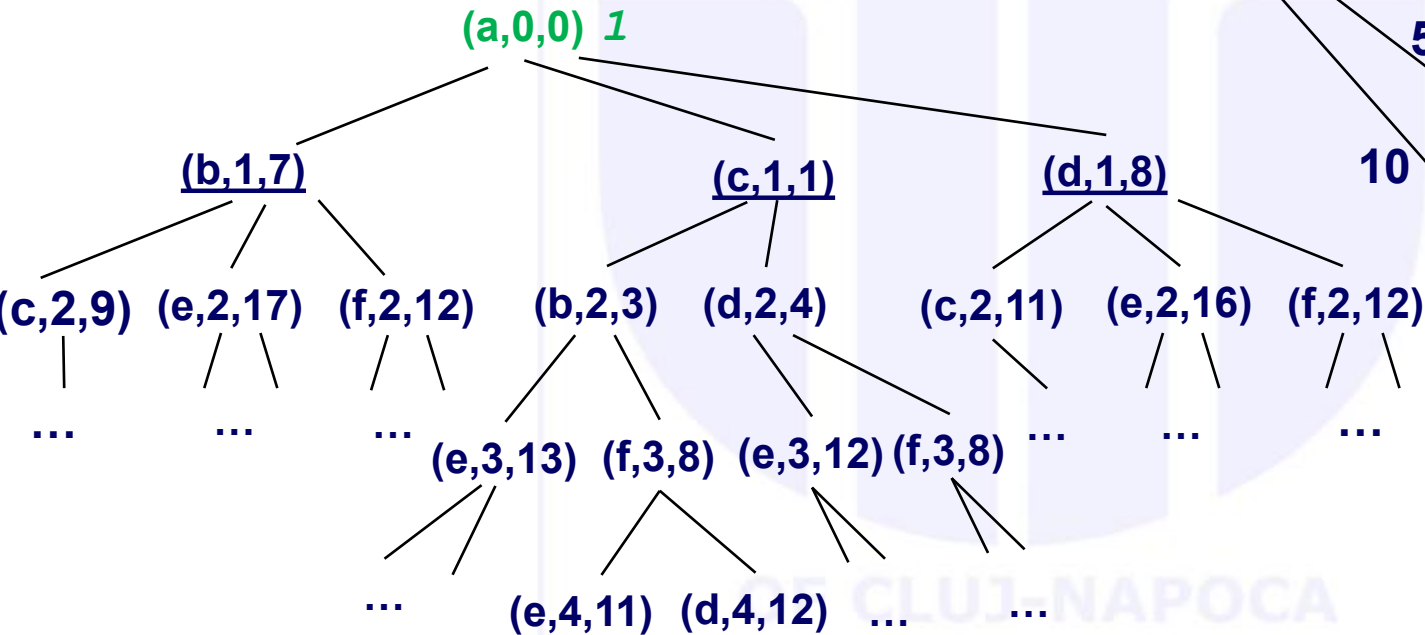
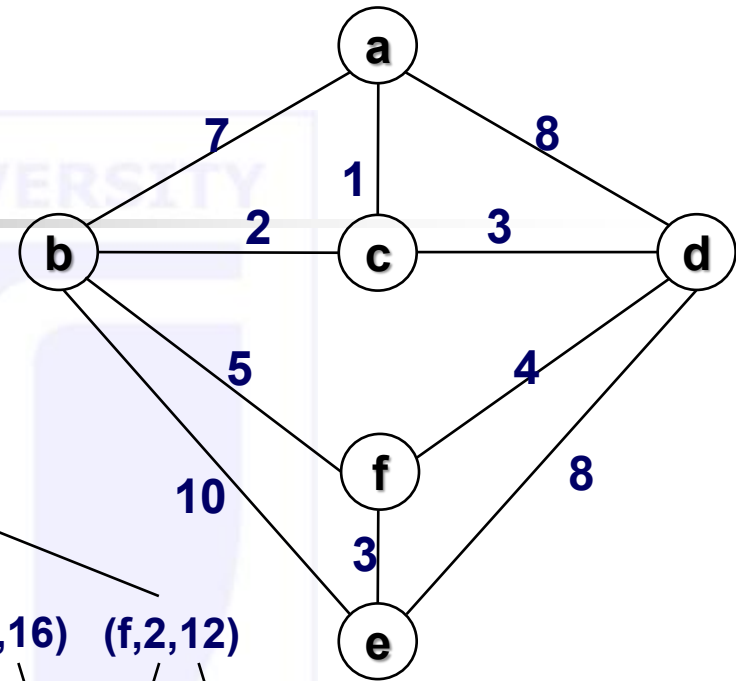


... expand first element in Cand

Cand = $[[n(a, 0, 0)]]$

Exp = $[[$

Example – traversal



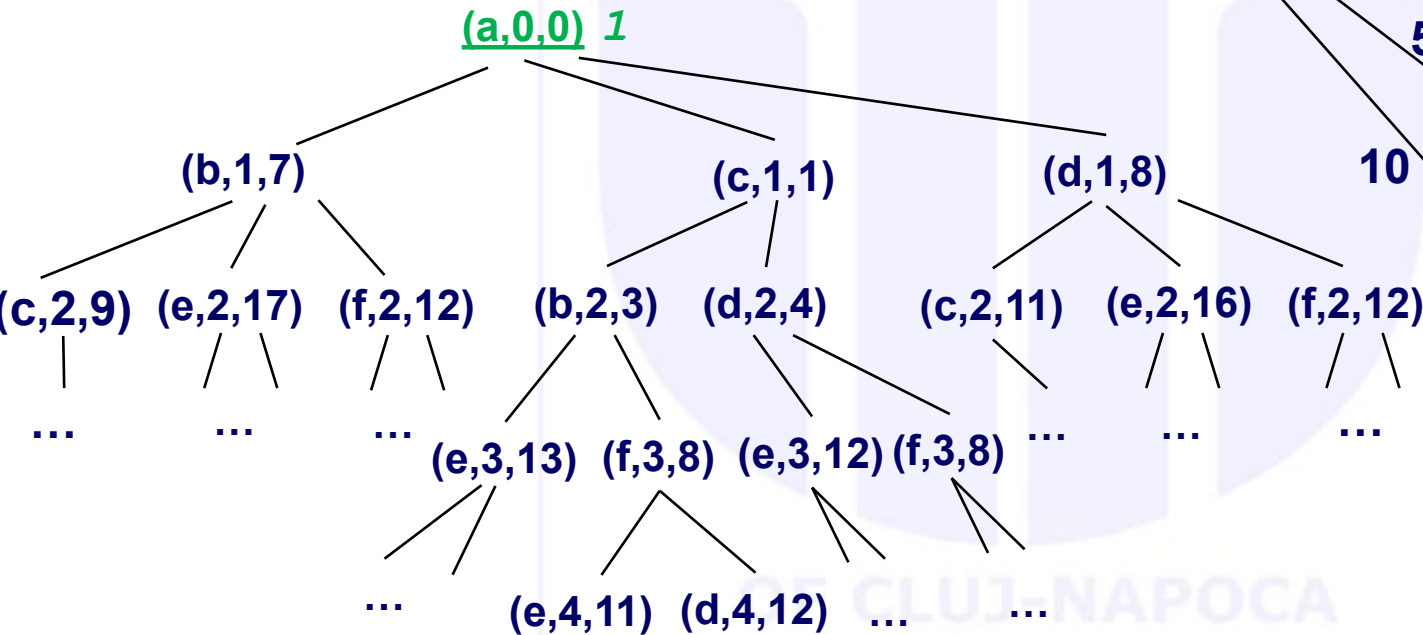
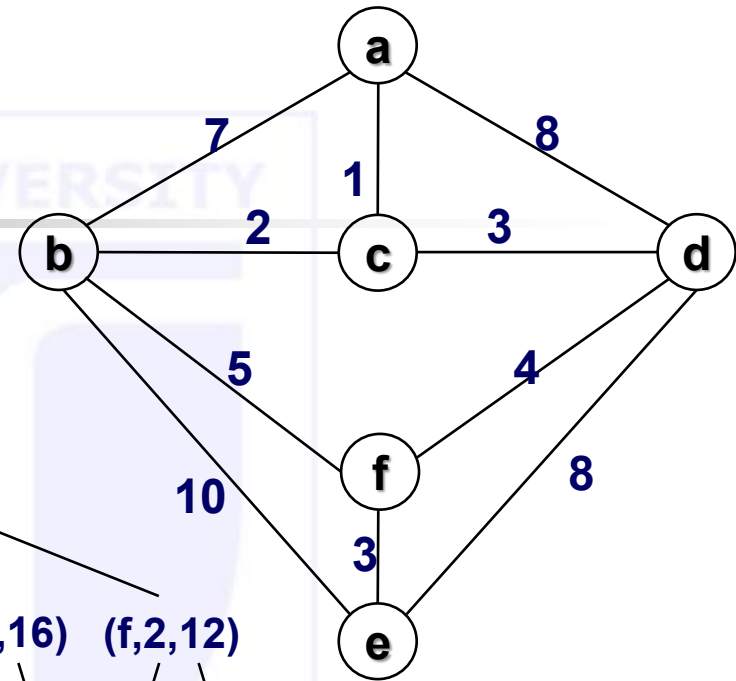
... after expanding $[n(a, 0, 0)]$

Cand = [
 $[n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$

]

Exp = []

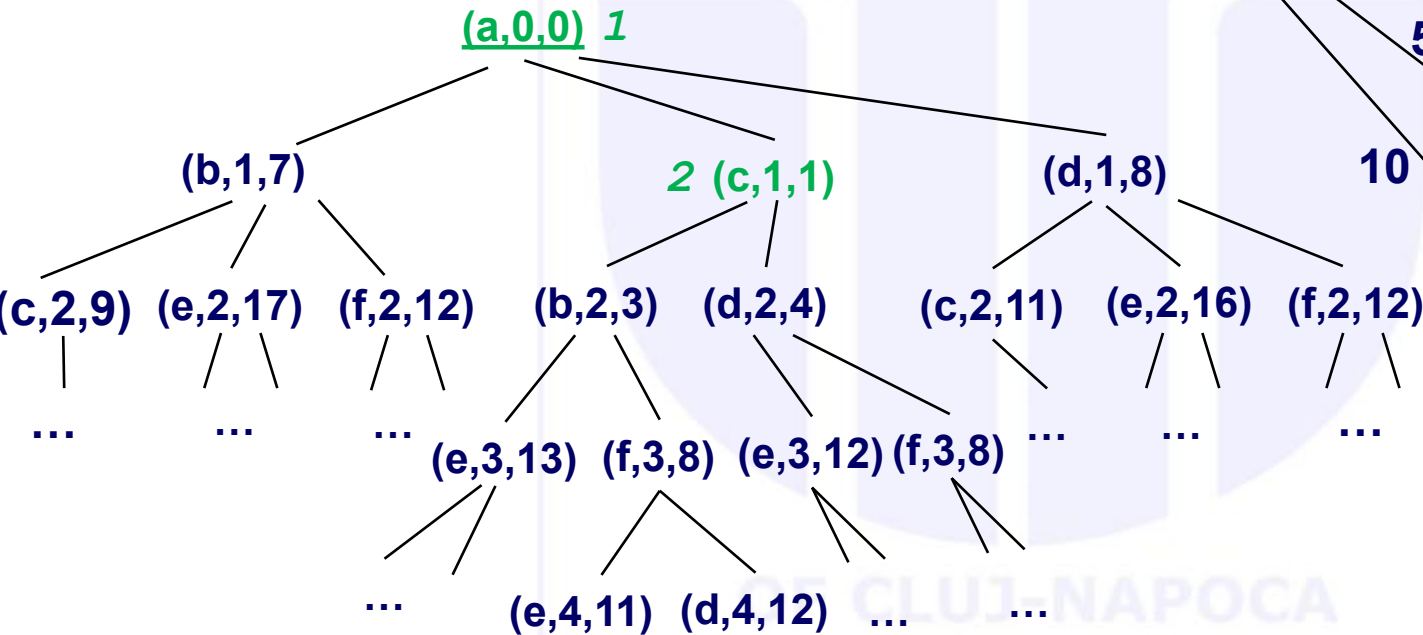
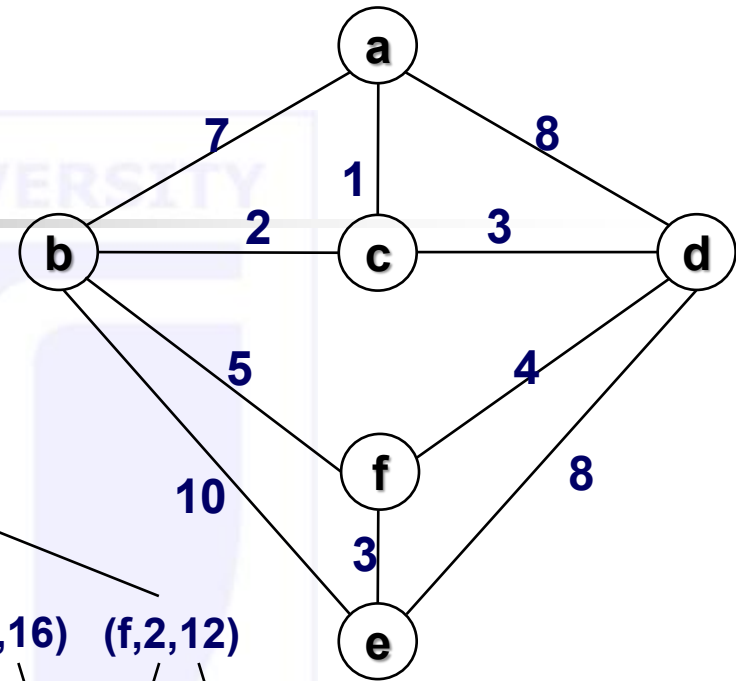
Example – traversal



... recursive call search

```
Cand = [
    [n(c, 1, 1), n(a, 0, 0)],
    [n(b, 1, 7), n(a, 0, 0)],
    [n(d, 1, 8), n(a, 0, 0)]
]
Exp = [[n(a, 0, 0)]]
```

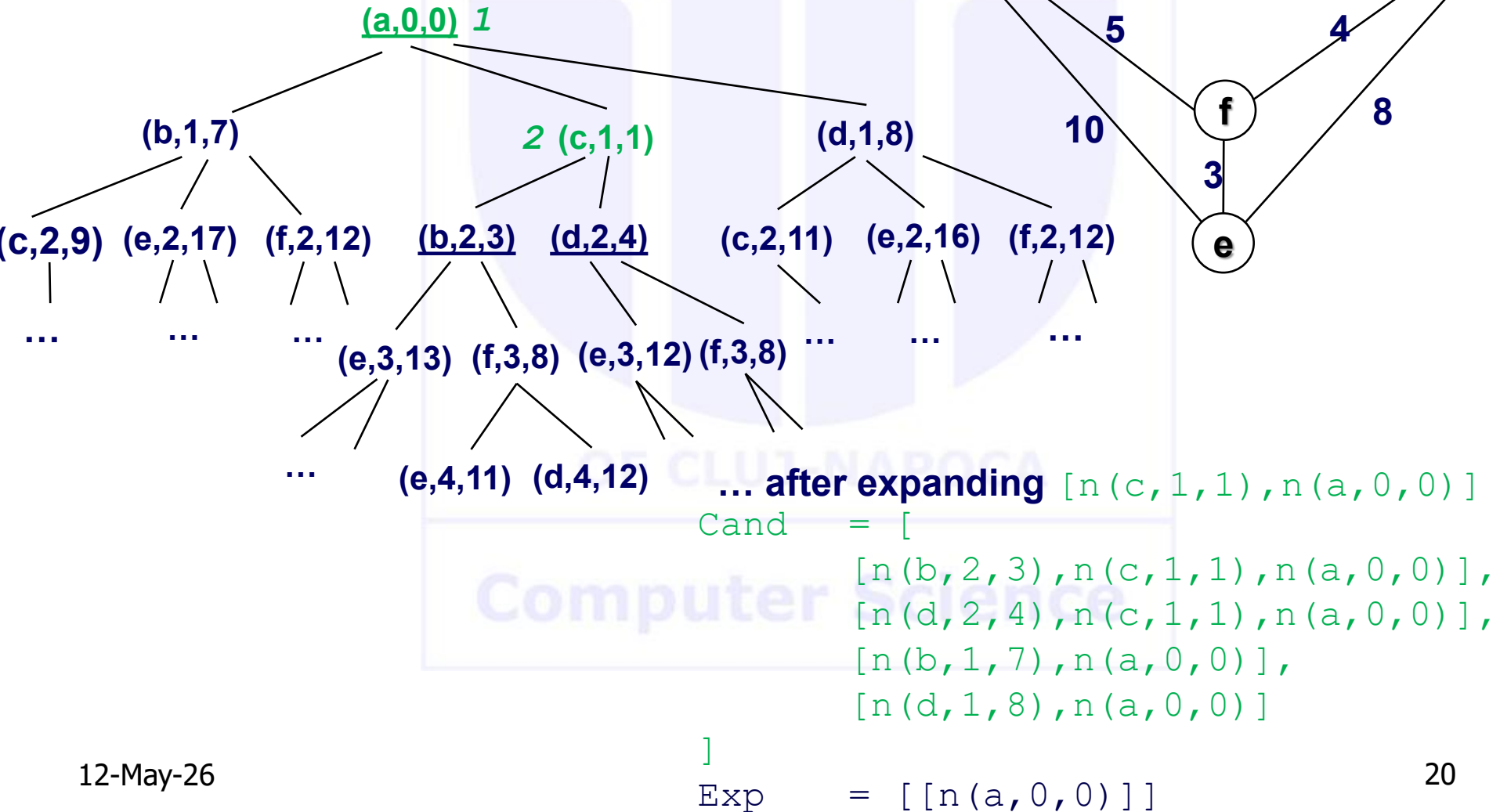
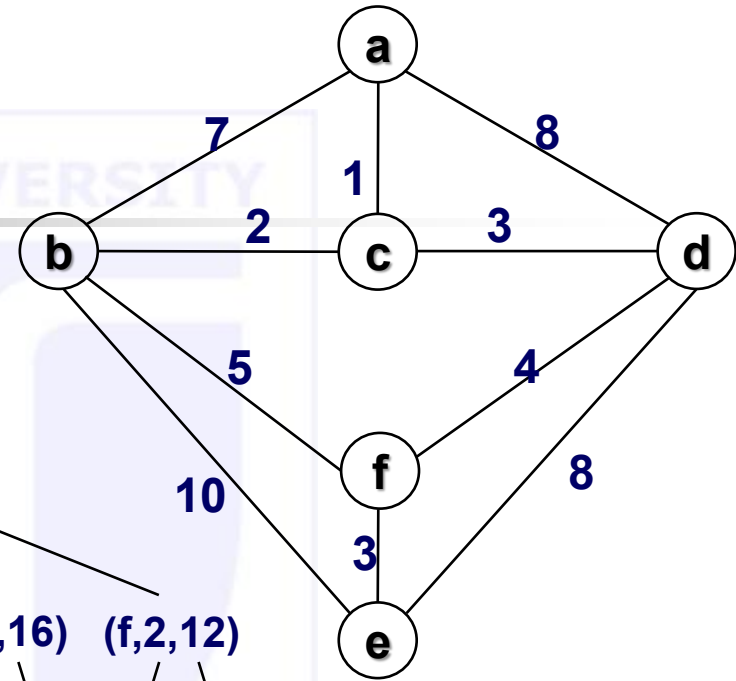
Example – traversal



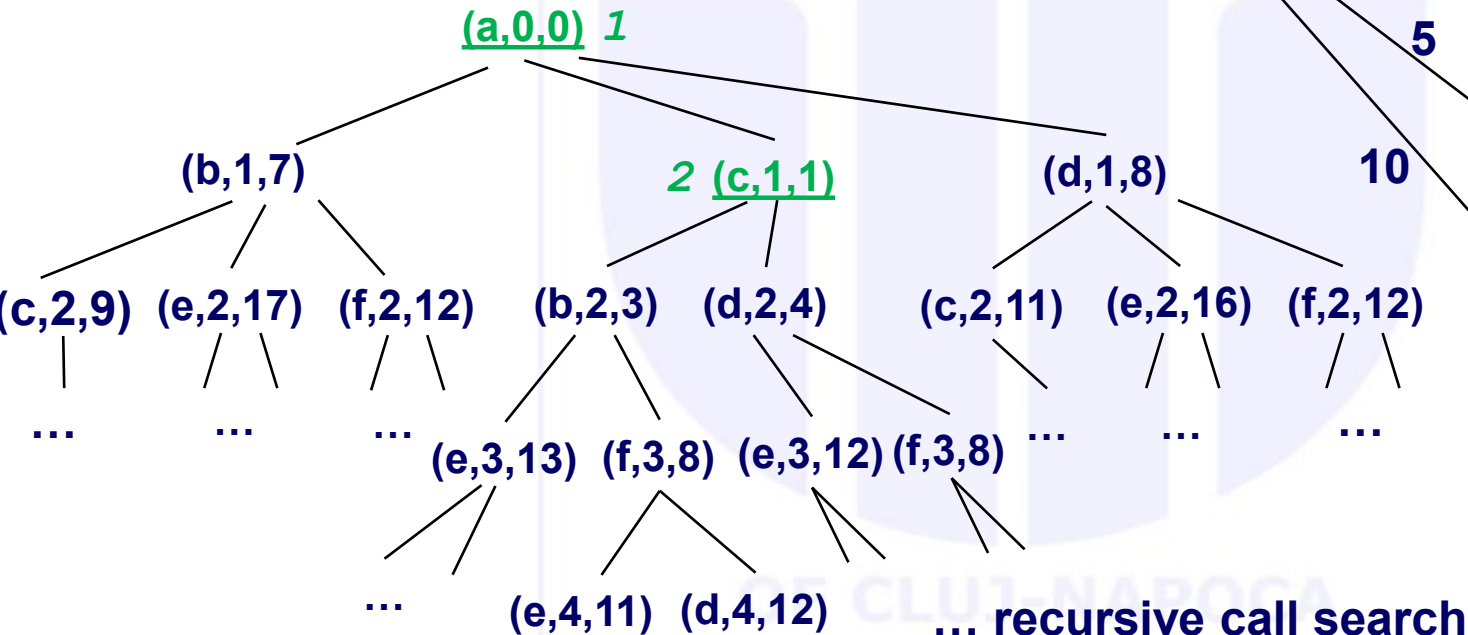
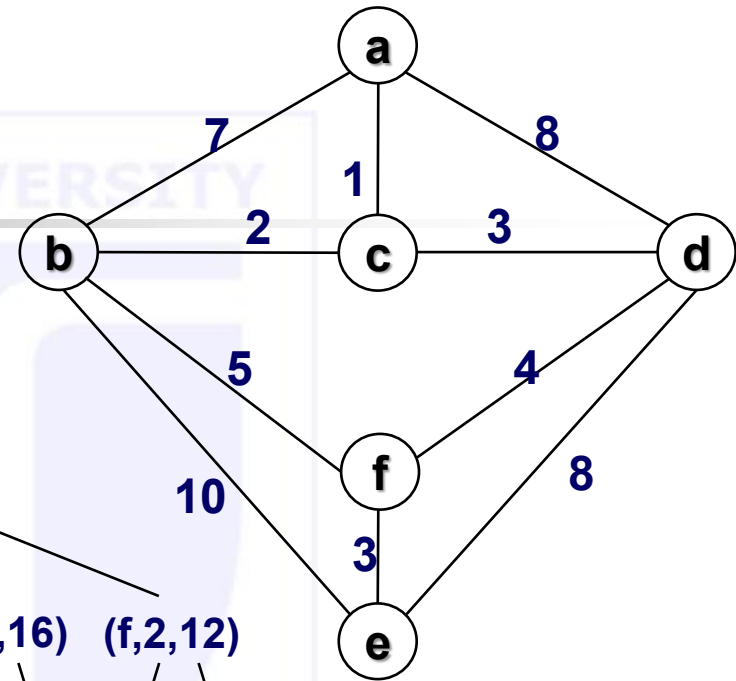
... this will be expanded next

Cand = [
 $[n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$
]
 Exp = $[[n(a, 0, 0)]]$

Example – traversal



Example – traversal

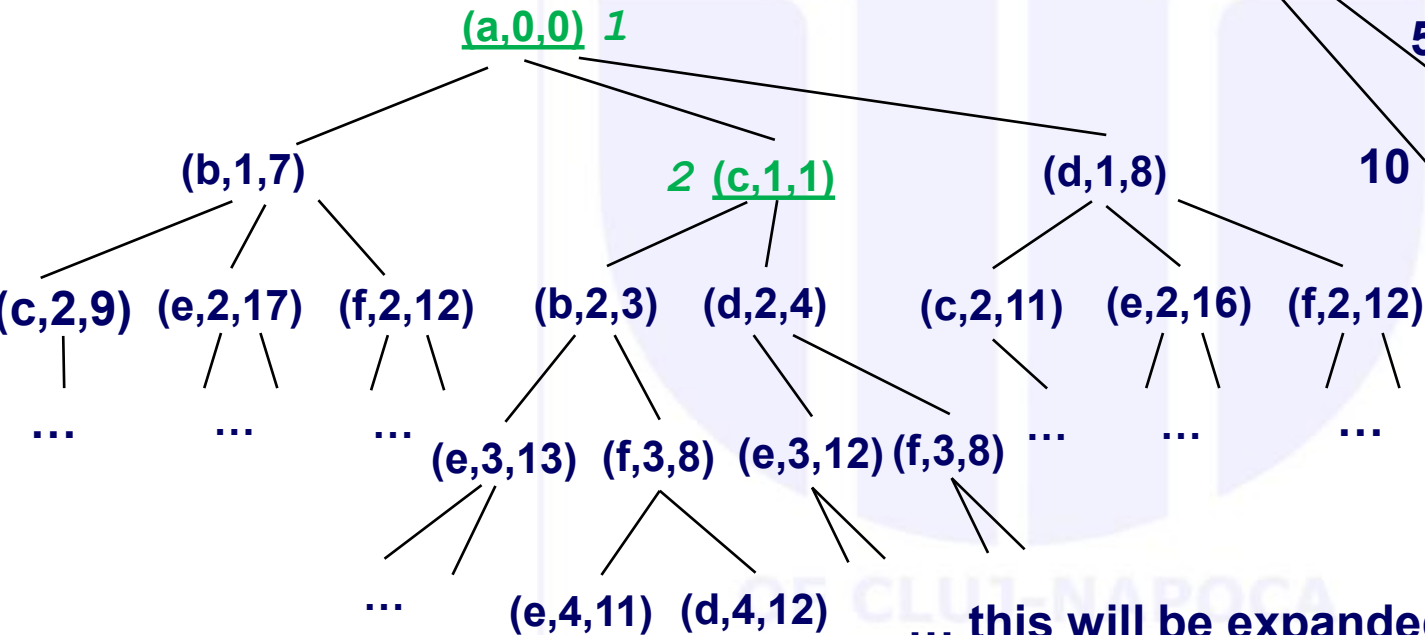
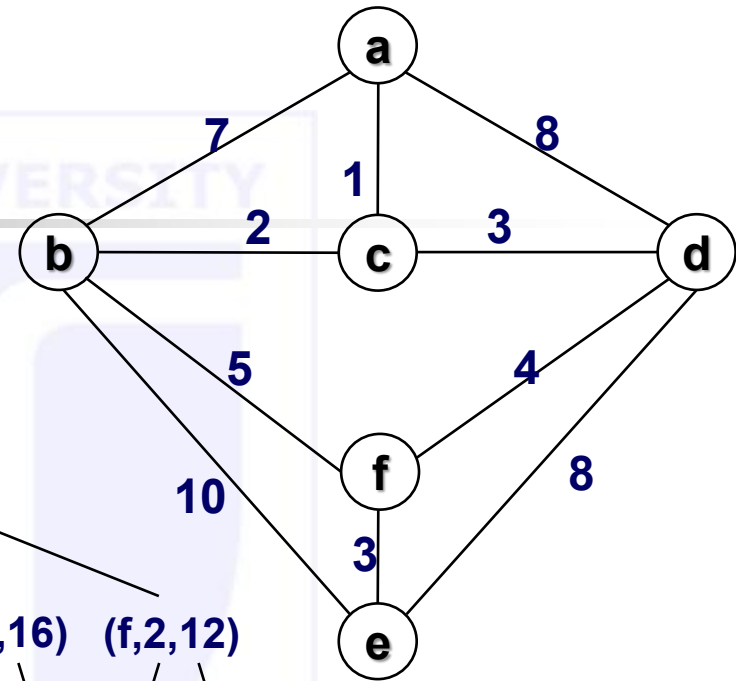


... recursive call search

Cand = [
 $[n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(d, 2, 4), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$
 $]$

Exp = $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)^{21}]]$

Example – traversal



... this will be expanded next

Cand

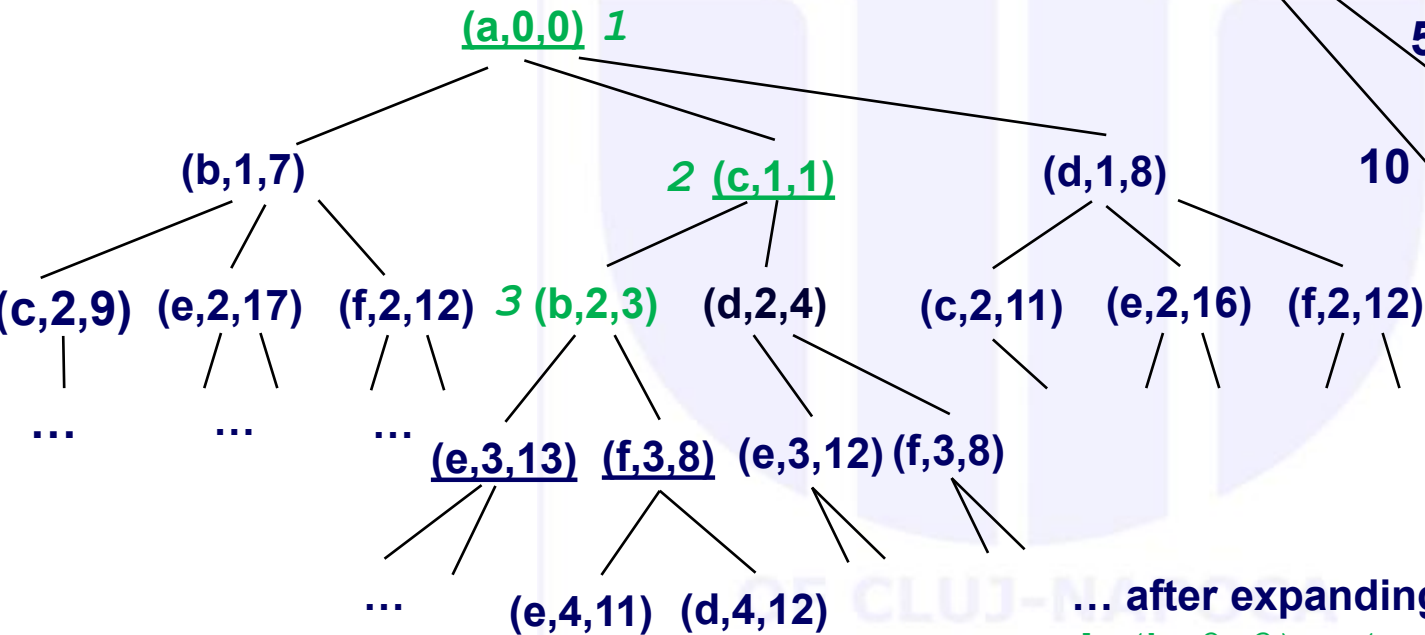
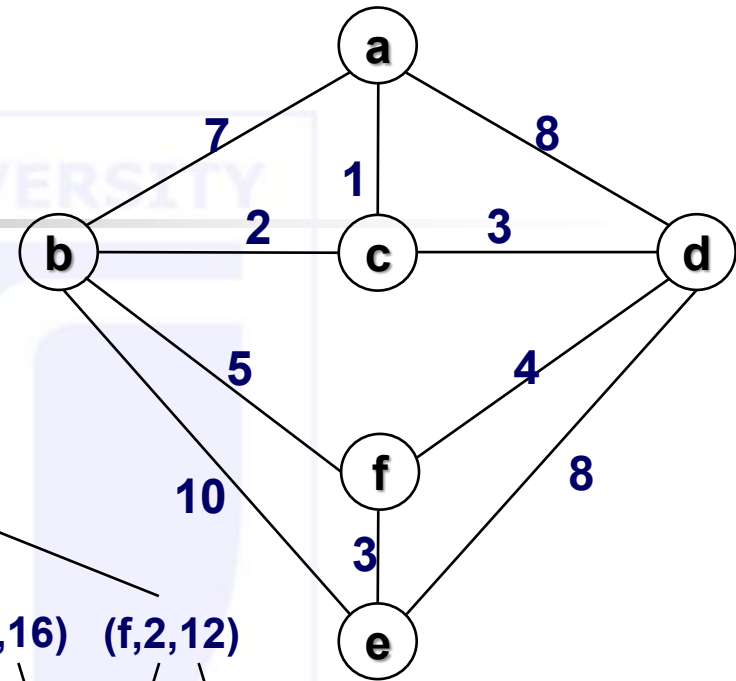
= [
 [n(b,2,3), n(c,1,1), n(a,0,0)],
 [n(d,2,4), n(c,1,1), n(a,0,0)],
 [n(b,1,7), n(a,0,0)],
 [n(d,1,8), n(a,0,0)]

]

Exp

= [[n(c,1,1), n(a,0,0)], [n(a,0,0)²²]]

Example – traversal



... after expanding

$[n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$

Cand

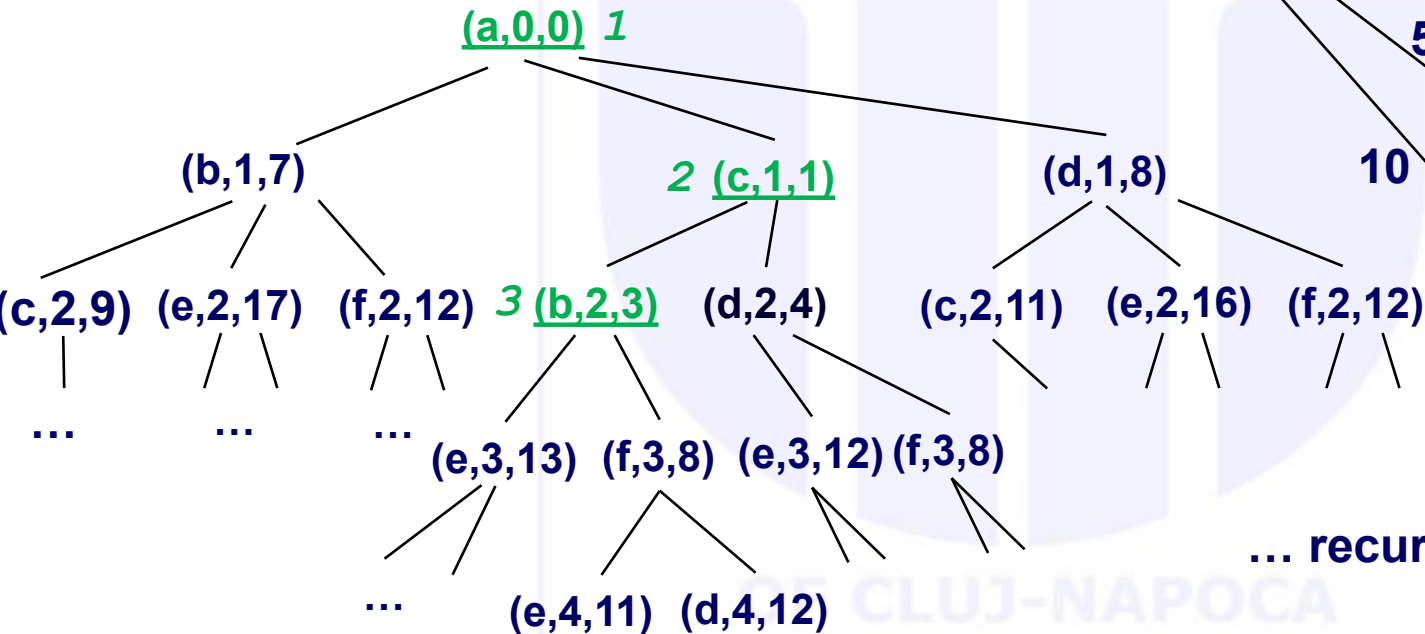
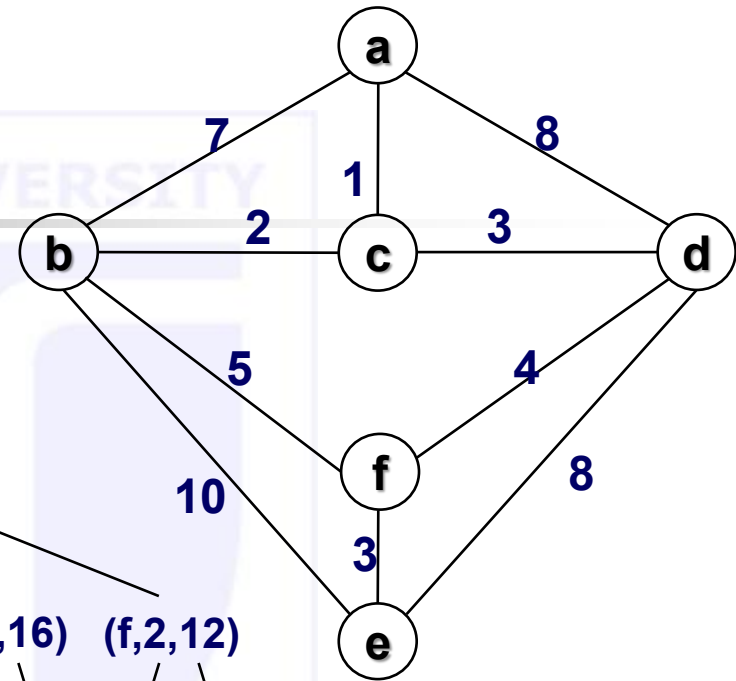
= [
 $[n(d, 2, 4), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$,
 $[n(f, 3, 8), n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(e, 3, 13), n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$

]

Exp

= $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)]]$ ²³

Example – traversal

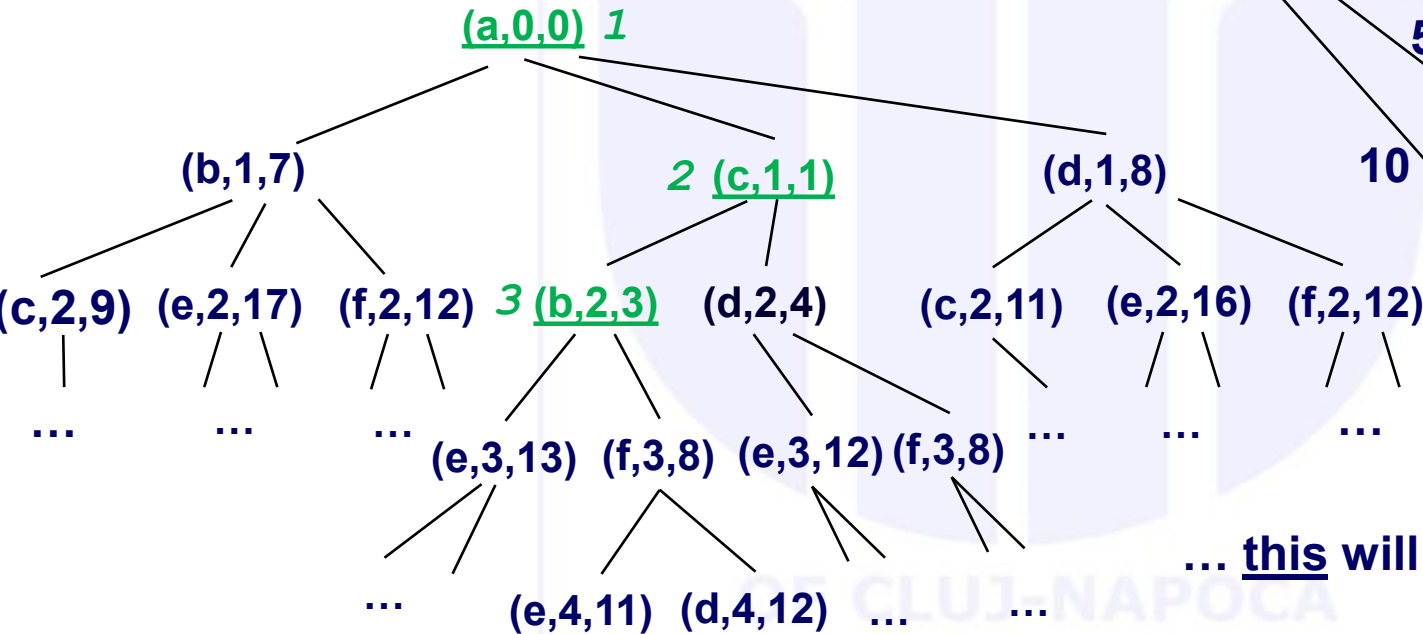
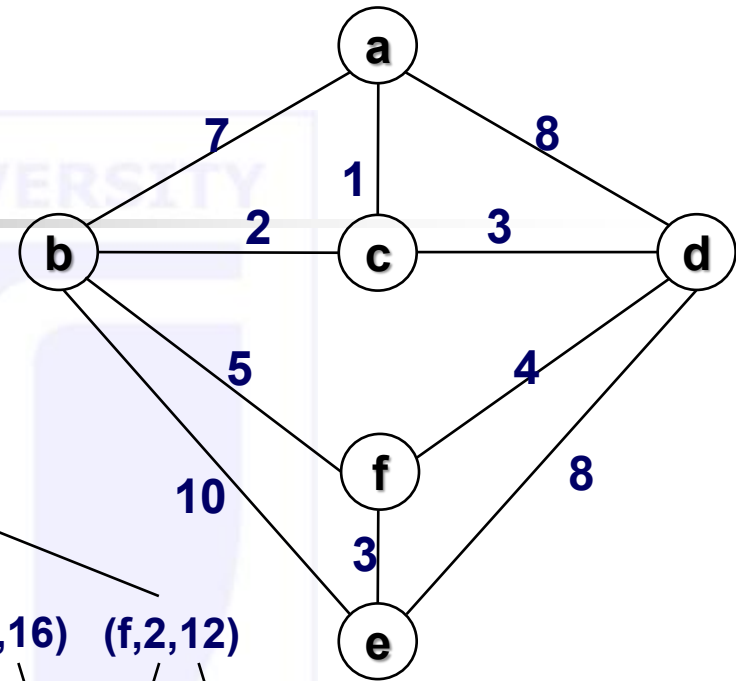


... recursive call search

```
Cand = [
    [n(d,2,4), n(c,1,1), n(a,0,0)],
    [n(b,1,7), n(a,0,0)],
    [n(d,1,8), n(a,0,0)],
    [n(f,3,8), n(b,2,3), n(c,1,1), n(a,0,0)],
    [n(e,3,13), n(b,2,3), n(c,1,1), n(a,0,0)]
]

Exp = [[n(b,2,3), n(c,1,1), n(a,0,0)],
        [n(c,1,1), n(a,0,0)],
        [n(a,0,0)]]
```

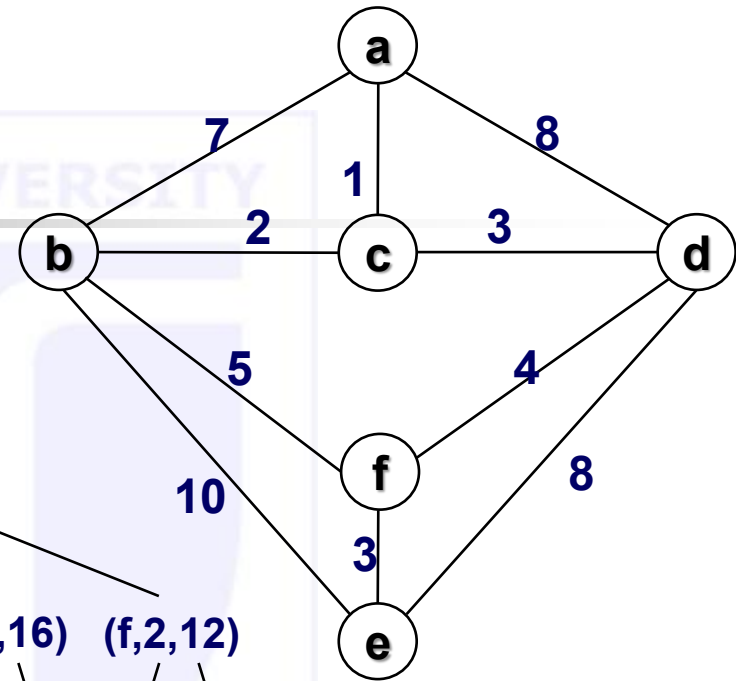
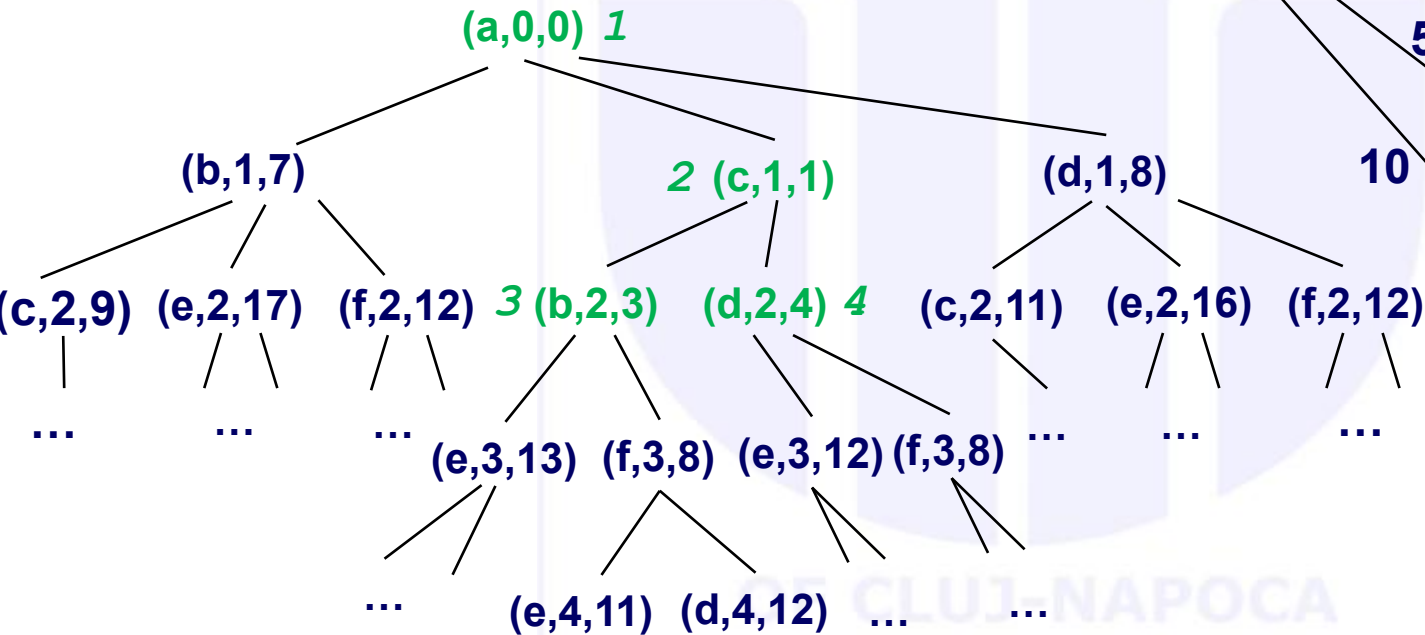

Example – traversal



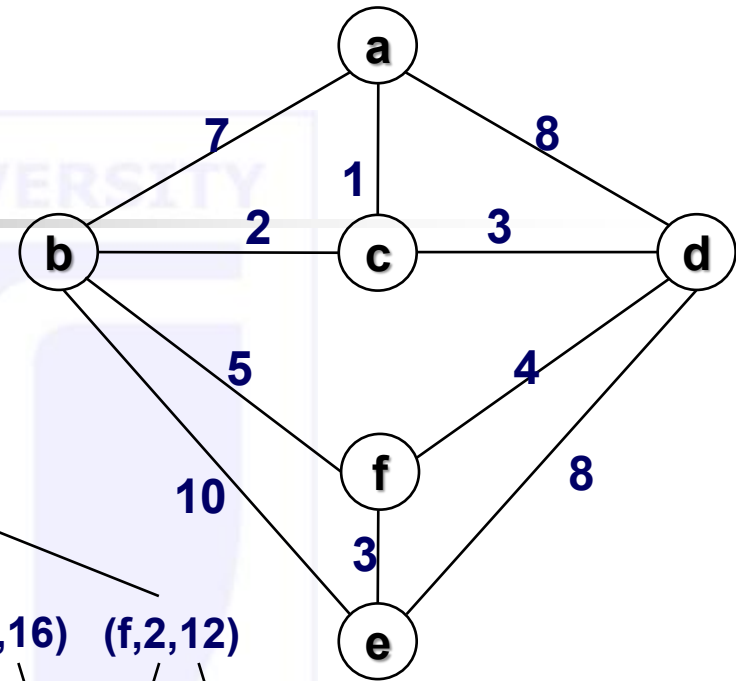
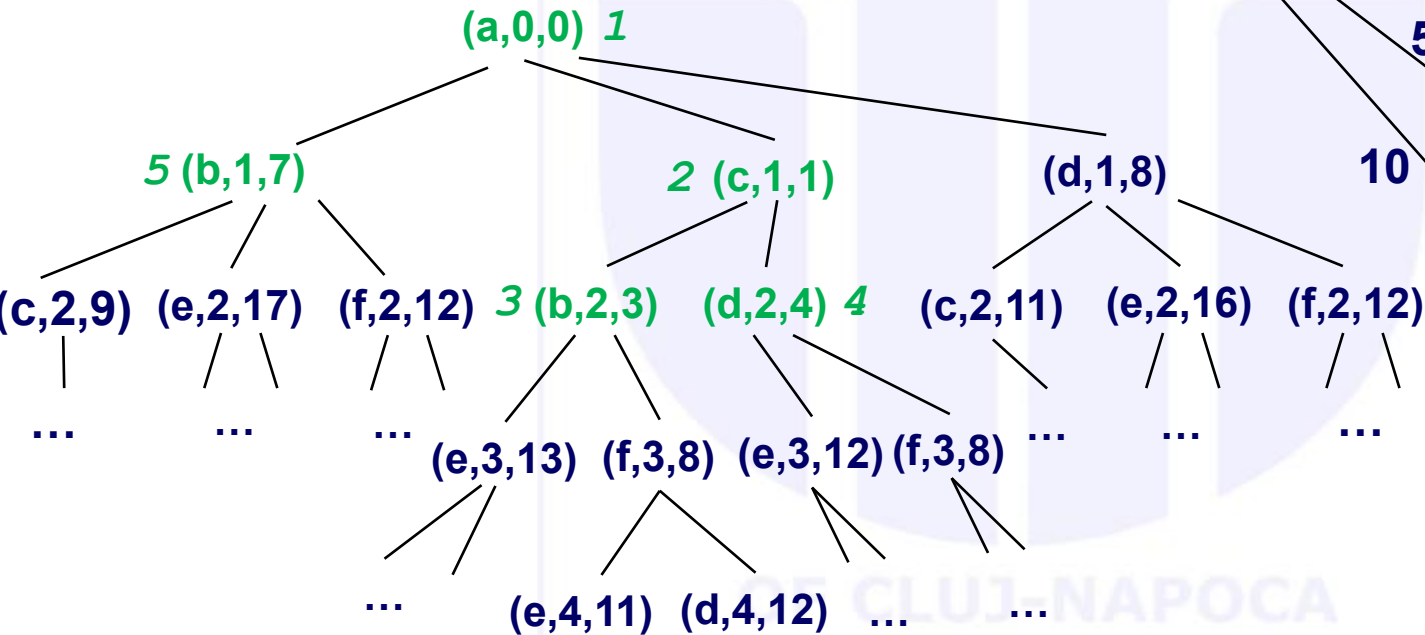
... this will be expanded next

Cand = [
 $\frac{[n(d,2,4), n(c,1,1), n(a,0,0)]}{[n(b,1,7), n(a,0,0)]},$
 $[n(d,1,8), n(a,0,0)],$
 $[n(f,3,8), n(b,2,3), n(c,1,1), n(a,0,0)],$
 $[n(e,3,13), n(b,2,3), n(c,1,1), n(a,0,0)]$
 $]$
 Exp = $[[n(b,2,3), n(c,1,1), n(a,0,0)],$
 $[n(c,1,1), n(a,0,0)],$ $[n(a,0,0)]]$

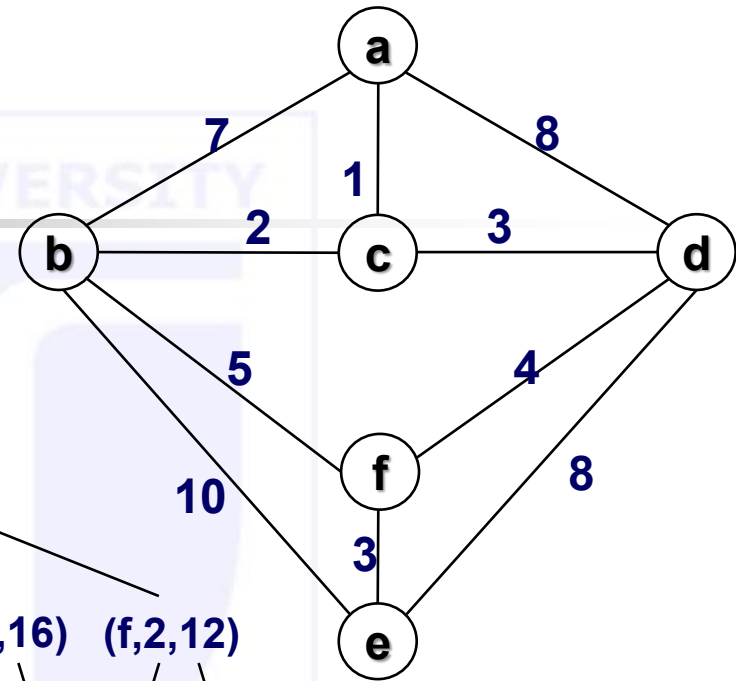
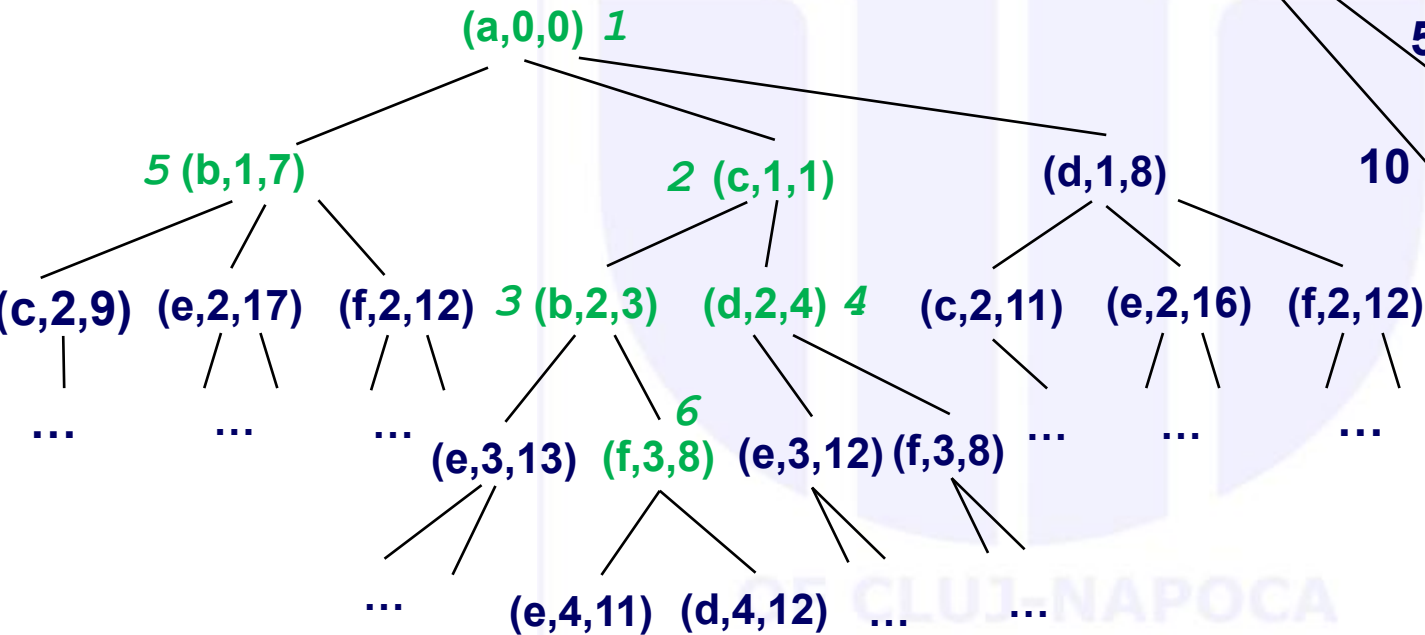
Example – traversal



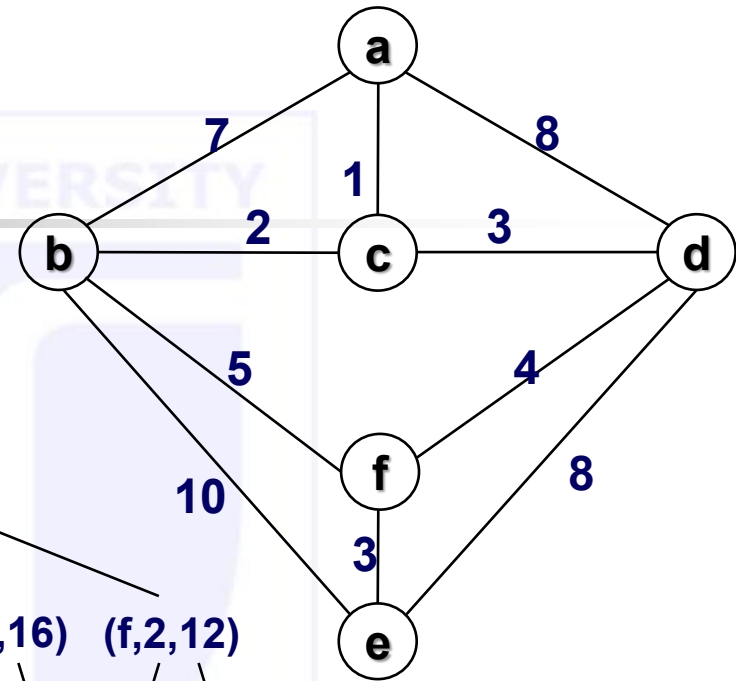
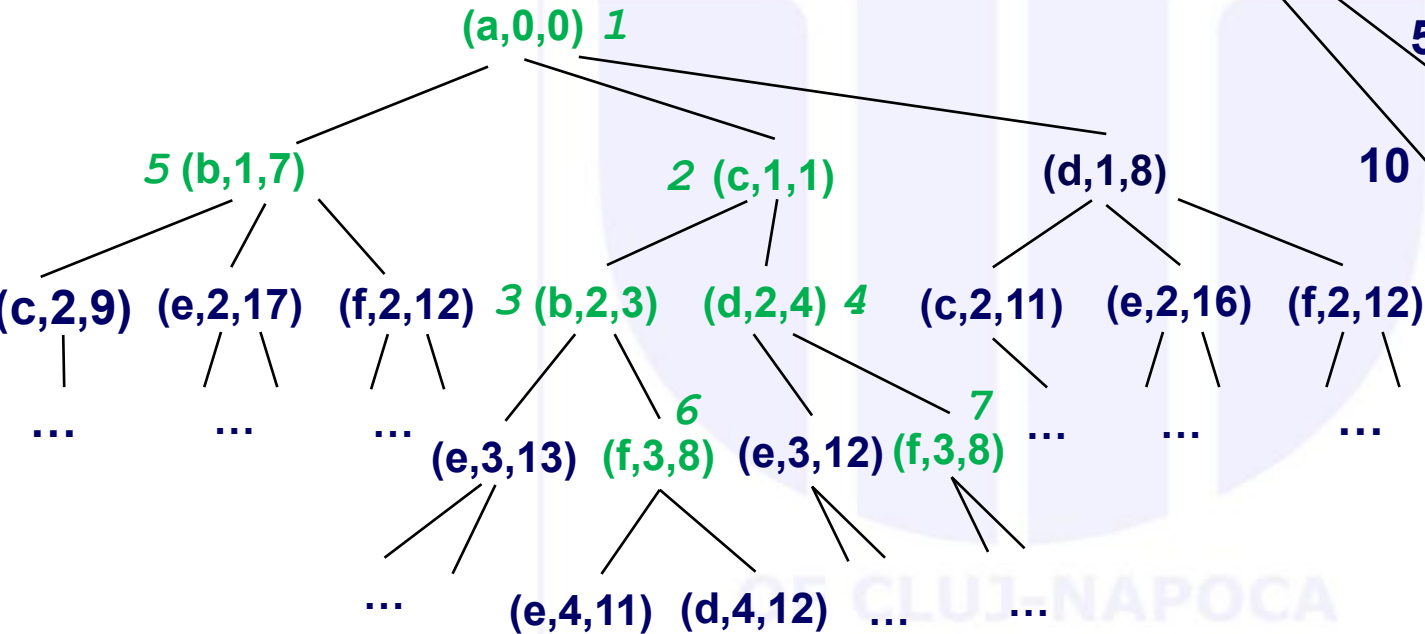
Example – traversal



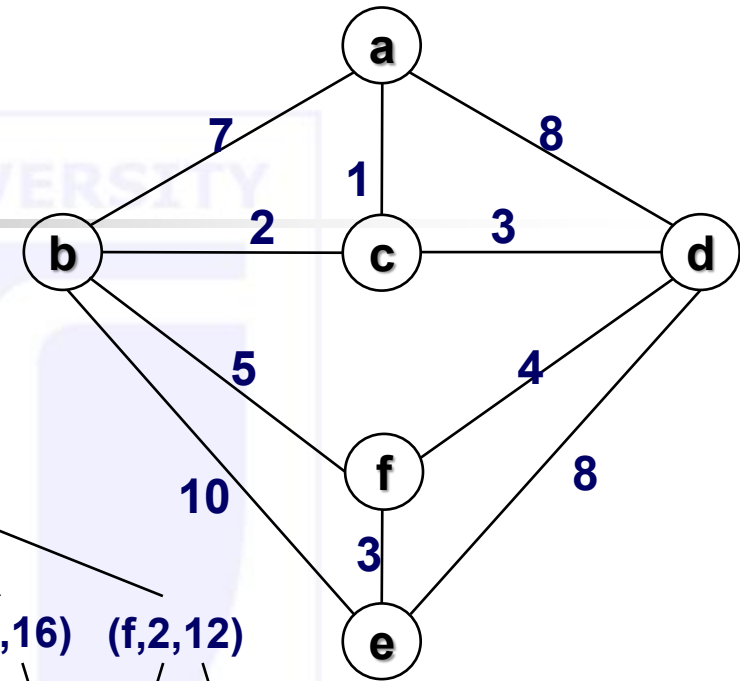
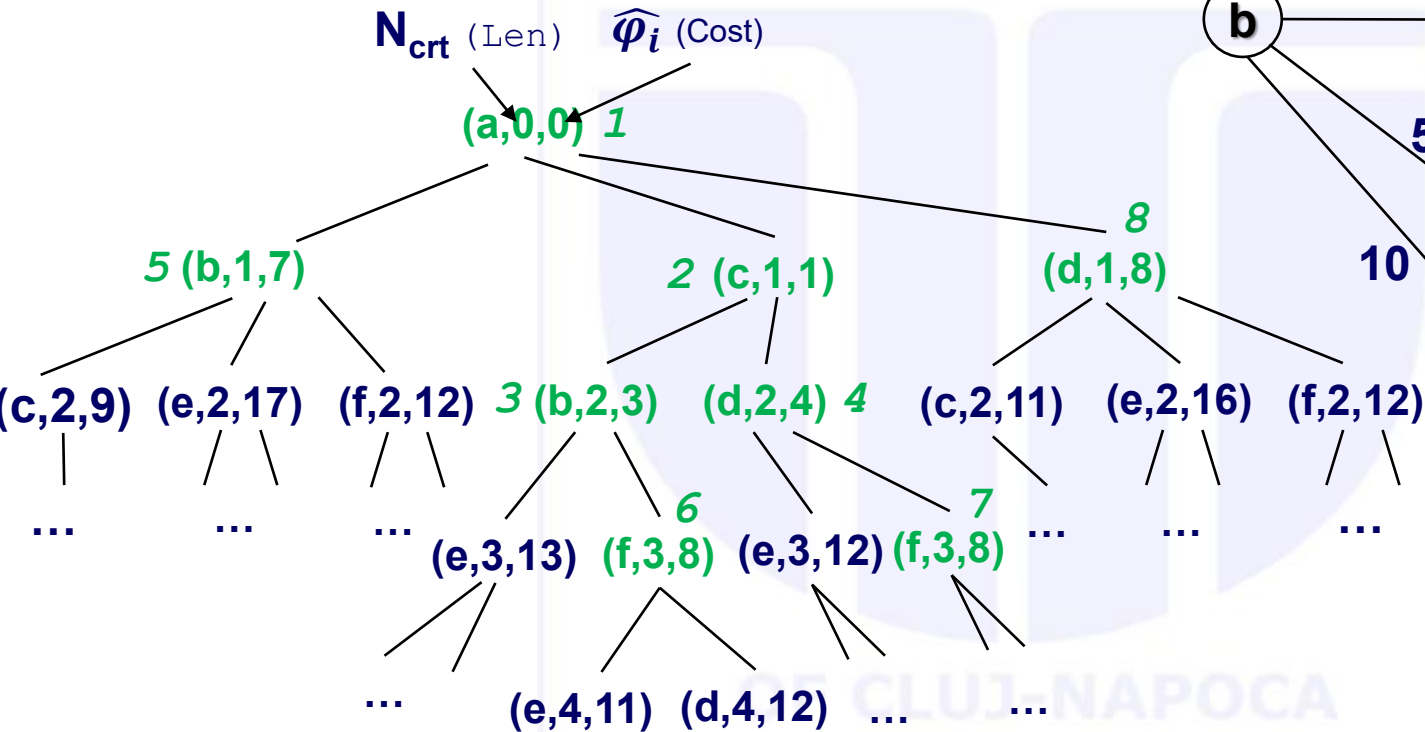
Example – traversal



Example – traversal



Example – traversal



... etc.